UNIVERSITY OF LEEDS

COMP1850 Programming

Week 7, Session 1

Introduction to Testing

UNIVERSITY OF LEEDS

Recap

Previously, we covered

- How to define a function in Python
- How to define a class in Python

UNIVERSITY OF LEEDS

Today's Aims

- Learn how to do **unit testing** of functions and methods in Python
- Get experience with using the `pytest` framework

Unit Testing

- **Unit tests** are tests written for a small part of the system, which is tested **in isolation** from other parts
- A developer typically creates unit tests for they code they themselves have written
- These tests should be written at the same time as (or even before) the code that needs to be tested

UNIVERSITY OF LEEDS

How Do We Test?

Consider a function `comfort_level()` that takes a temperature in °C (a `float`) as the only argument. It is supposed to return

- `"COLD"` for temperatures below 15.0°C
- `"HOT"` for temperatures above 25.0°C
- `"OK"` for temperatures within the range 15.0°C – 25.0°C

Is it possible to test this function exhaustively, with all possible inputs?

Equivalence Classes

An **equivalence class** is a set of possible inputs for which a function is expected to behave in the same way.

`comfort_level()` has three equivalence classes: one for each of its possible outputs.

We can test each equivalence class using

- A value at its lower boundary (if there is one)
- A value at its upper boundary (if there is one)
- A ‘typical’ value, somewhere between these boundaries

UNIVERSITY OF LEEDS

Task 1: Values for Equivalence Classes

`week1.7/session1/task1`

- Edit `tests.md` in each folder and identify values that can be used to test each of the equivalence classes
- Use the Markdown Preview button in VS Code to check that the table is formatted properly

Implementing a Test

Three Steps:

1. **Arrange**: set up the **preconditions** for the test
2. **Act**: call the function that we are testing
3. **Assert**: make **assertions** about the results of calling the function

```
assert boolean-expression
```

UNIVERSITY OF LEEDS

Example: `comfort_level()`

```
temp = 20.0                          # Arrange
level = comfort_level(temp)          # Act
assert level == "OK"                 # Assert
```

We can collapse these statements into a single statement:

```
assert comfort_level(20.0) == "OK"
```

UNIVERSITY OF LEEDS

Putting it all together…

```
from weather import comfort_level

def test_cold():
    assert comfort_level(    ) == "COLD"
    assert comfort_level(    ) == "COLD"

def test_hot():
    assert comfort_level(    ) == "HOT"
    assert comfort_level(    ) == "HOT"
```



Put your values in here!

Example: Point Constructor

```
from geometry import Point

def test_point_constructor():
    p = Point(5, -2)
    assert p.x == 5
    assert p.y == -2
```

Task 2: Writing Test Functions

UNIVERSITY OF LEEDS

`week1.7/session1/task2`

Complete the missing implementation for all test functions in each folder

Running Tests

Could write a program that imports and runs the test functions, or we could do it interactively in the Python interpreter:

```
>>> from test_comfort import *
>>> test_cold()
>>> test_hot()
>>> test_ok()
```

Is there a better way?...

UNIVERSITY OF LEEDS

pytest

- A popular unit testing framework for Python code
- Includes tools to simplify writing of tests
- Can find your tests automatically, and run what it finds
- Reports test results in a useful way

(See https://docs.pytest.org/ for more information)

UNIVERSITY OF LEEDS

Virtual Environments (1)

It's a good idea to create a **virtual environment** whenever you want to install Python packages (e.g., pytest).

Make sure you are in the `week1.7/session1/tasks` directory of your repository and enter this command:

```
python -m venv env
```



Name of environment (and name of subdirectory that will be created)

Virtual Environments (2)

UNIVERSITY OF LEEDS

Make sure that Git is ignoring the virtual environment subdirectory (repository's `.gitignore` file should have a line for `env`)

Activate the environment:

```
source env/bin/activate
```

(You can leave it at any time by entering `deactivate`)

UNIVERSITY OF LEEDS

Installing pytest

With the virtual environment active, use `pip` to install pytest:

```
pip install pytest
```

Check that you are now able to run pytest, with

```
pytest --version
```

Running pytest

- Enter `pytest` followed by the name of a `.py` file containing tests
- … or just enter `pytest` and let it find the tests itself
- Use the `–v` option to run verbosely and see more detailed results
- See https://docs.pytest.org/ for more information

Test Discovery

Test are easily discoverable by pytest if you

- Write them as functions whose names begin with `test_`
- Put those functions inside `.py` files whose names begin with `test_`

If you have multiple `.py` files containing tests, it is a good idea to put them in their own subdirectory (e.g., named `test` or `tests`)

pytest Output

- By default: `.` or `F` printed for each test function it finds (F = Failed)
- With `-v`: test functions are listed, with ‘PASSED’ or ‘FAILED’
- Full details of any failing tests are displayed, showing what was expected and what was actually seen in a failed assertion



UNIVERSITY OF LEEDS

Task 3: Run pytest

`week1.7/session1/task3`

1. Install pytest into a virtual environment, by following the steps outlined earlier
2. Use pytest to run the tests implemented for Task 2
3. Use the test output to identify and fix any problems
4. Rerun the tests to verify that they all now pass

UNIVERSITY OF LEEDS

Comparing floats with pytest

`pytest` provides a utility to compare floating-point values that may have small rounding errors

1. Import `pytest` module
2. Use `pytest.approx( )` to compare values

Example

```
import pytest

def test_sum():
    assert (0.1 + 0.2) == pytest.approx(0.3)
```



UNIVERSITY OF LEEDS

Test Expected Exceptions with pytest

To write assertions about raised exceptions:

1. Import `pytest`
2. Use `pytest.raises()` as a context manager

Example 1

```
import pytest

def myfunc():
    raise ValueError("Exception 123 raised")

def test_match():
    with pytest.raises(ValueError) as excinfo
        myfunc()
    assert str(excinfo.value) == "Exception 123 raised"
```

Example 2

```
import pytest
from money import Money


def test_invalid_type_pound():
    with pytest.raises(TypeError) as excinfo:
        Money(1.2, 99)
    assert str(excinfo.value) == "pounds must be an integer"
```

UNIVERSITY OF LEEDS

Task 4: Test Expected Exceptions

`week1.7/session1/task4`

1. Complete the implementation for testing the `__init__( )` and `__add__()` methods in `Money` class
2. Use `pytest` to run the tests implemented
3. Use the test output to identify and fix any problems
4. Rerun the tests to verify that they all now pass

UNIVERSITY OF LEEDS

Summary

We have

- Introduced unit testing
- Implemented unit tests with `assert`
- Introduced the pytest unit testing framework

UNIVERSITY OF LEEDS

Next session

- Debugging in Python